Load Dataset

```
!pip install --upgrade pip

!pip install statsmodels pmdarima
import warnings

# Ignore Warnings
warnings.filterwarnings("ignore")
```

```
Requirement already satisfied: pip in /usr/local/lib/python3.11/dist-packag
Collecting pip
  Downloading pip-25.1.1-py3-none-any.whl.metadata (3.6 kB)
Downloading pip-25.1.1-py3-none-any.whl (1.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.8/1.8 MB 10.5 MB/s eta 0:00:0
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.1.2
    Uninstalling pip-24.1.2:
      Successfully uninstalled pip-24.1.2
Successfully installed pip-25.1.1
Requirement already satisfied: statsmodels in /usr/local/lib/python3.11/dis
Collecting pmdarima
  Downloading pmdarima-2.0.4-cp311-cp311-manylinux_2_17_x86_64.manylinux201
Requirement already satisfied: numpy<3,>=1.22.3 in /usr/local/lib/python3.1
Requirement already satisfied: scipy!=1.9.2,>=1.8 in /usr/local/lib/python3
Requirement already satisfied: pandas!=2.1.0,>=1.4 in /usr/local/lib/python
Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.11/di
Requirement already satisfied: packaging>=21.3 in /usr/local/lib/python3.11
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.11/di
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/loc
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3
Requirement already satisfied: urllib3 in /usr/local/lib/python3.11/dist-pa
Requirement already satisfied: setuptools!=50.0.0,>=38.6.0 in /usr/local/li
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/pyt
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.11/di
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.11/
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.11/dist-p
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/pytho
Downloading pmdarima-2.0.4-cp311-cp311-manylinux_2_17_x86_64.manylinux2014_
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 2.2/2.2 MB 32.7 MB/s eta 0:00:0
Installing collected packages: pmdarima
Successfully installed pmdarima-2.0.4
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose
from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.stattools import adfuller
from sklearn.metrics import mean_squared_error, mean_absolute_error

import kagglehub

# Download latest version
path = kagglehub.dataset_download("vijayvvenkitesh/microsoft-stock-time-series-a

import os

# Lihat isi folder hasil download
files = os.listdir(path)
print("Files in dataset directory:", files)

csv_path = os.path.join(path, "Microsoft_Stock.csv")
df = pd.read_csv(csv_path)
```

```
Files in dataset directory: ['Microsoft_Stock.csv']
```

```
# Display dataset info and description
print(df.info())
print(df.describe())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1511 entries, 0 to 1510
Data columns (total 6 columns):
 #   Column  Non-Null Count  Dtype  
---  ------  --------------  -----  
 0   Date    1511 non-null   object 
 1   Open    1511 non-null   float64
 2   High    1511 non-null   float64
 3   Low     1511 non-null   float64
 4   Close   1511 non-null   float64
 5   Volume  1511 non-null   int64  
dtypes: float64(4), int64(1), object(1)
memory usage: 71.0+ KB
None
              Open         High          Low        Close        Volume
count  1511.000000  1511.000000  1511.000000  1511.000000  1.511000e+03
mean    107.385976   108.437472   106.294533   107.422091  3.019863e+07
std      56.691333    57.382276    55.977155    56.702299  1.425266e+07
min      40.340000    40.740000    39.720000    40.290000  1.016120e+05
25%      57.860000    58.060000    57.420000    57.855000  2.136213e+07
50%      93.990000    95.100000    92.920000    93.860000  2.662962e+07
75%     139.440000   140.325000   137.825000   138.965000  3.431962e+07
max     245.030000   246.130000   242.920000   244.990000  1.352271e+08
```

```
df.head()
```

	Date	Open	High	Low	Close	Volume
0	4/1/2015 16:00:00	40.60	40.76	40.31	40.72	36865322
1	4/2/2015 16:00:00	40.66	40.74	40.12	40.29	37487476
2	4/6/2015 16:00:00	40.34	41.78	40.18	41.55	39223692
3	4/7/2015 16:00:00	41.61	41.91	41.31	41.53	28809375
4	4/8/2015 16:00:00	41.48	41.69	41.04	41.42	24753438

Next steps: Generate code with df | View recommended plots | New interactive sheet

EDA

```
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Load data (assuming the data is already in the dataframe 'df')
# If the data is in a CSV or another source, you'd use df = pd.read_csv() or si

# Ensure the 'Date' column is converted to datetime
df['Date'] = pd.to_datetime(df['Date'])
df.set_index('Date', inplace=True)

# Descriptive statistics
print("Descriptive Statistics:")
print(df.describe())

# Line plot to visualize the trend of Closing Price over time
plt.figure(figsize=(12, 6))
plt.plot(df.index, df['Close'], label='Closing Price', color='blue')
plt.title('Microsoft Stock Closing Price Over Time')
plt.xlabel('Date')
plt.ylabel('Closing Price ($)')
plt.legend()
plt.grid(True)
plt.show()

# Boxplot to check for outliers in the Closing Price
plt.figure(figsize=(8, 6))
plt.boxplot(df['Close'], vert=False)
plt.title('Box Plot of Microsoft Stock Closing Price')
plt.xlabel('Closing Price ($)')
plt.grid(True)
plt.show()

# Histogram of Closing Price distribution
plt.figure(figsize=(8, 6))
plt.hist(df['Close'], bins=20, color='skyblue', edgecolor='black')
plt.title('Distribution of Microsoft Stock Closing Prices')
plt.xlabel('Closing Price ($)')
plt.ylabel('Frequency')
plt.grid(True)
plt.show()

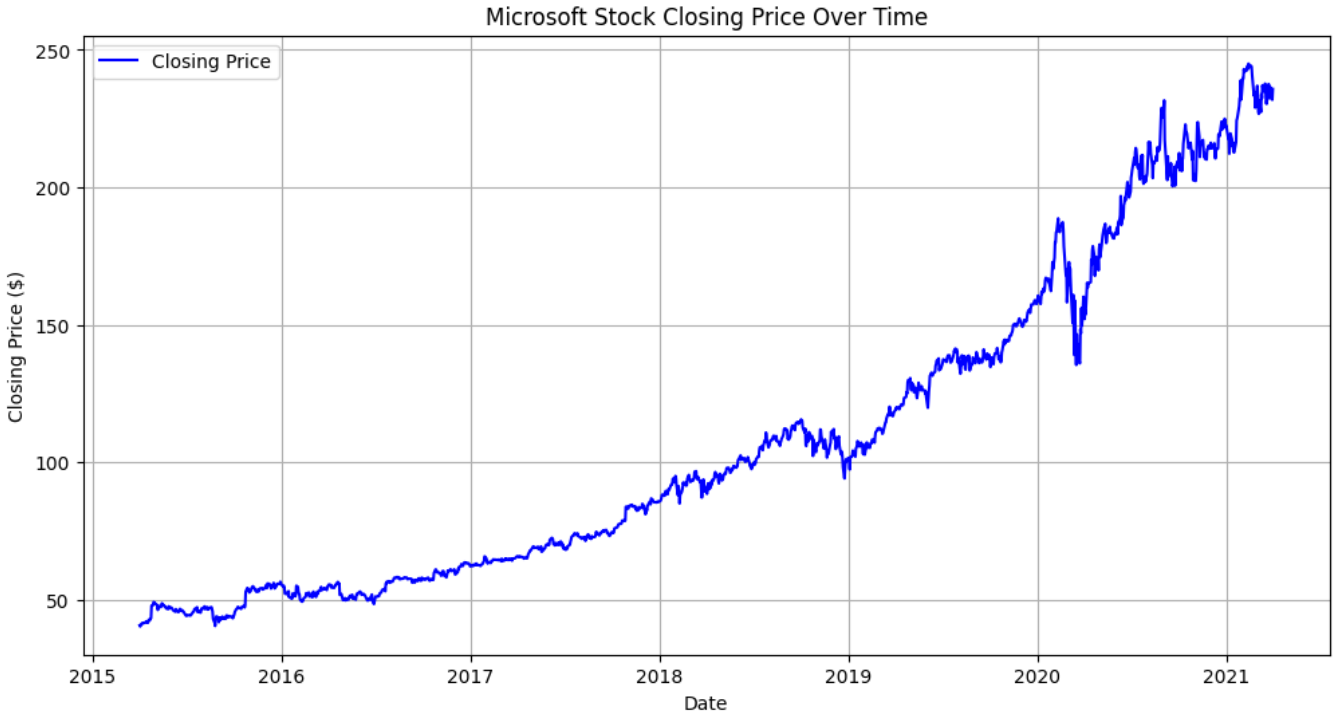
# Decompose the time series (using the 'Close' column to analyze trends, seasor
result = seasonal_decompose(df['Close'], model='additive', period=30)  # Adjust
result.plot()
plt.show()

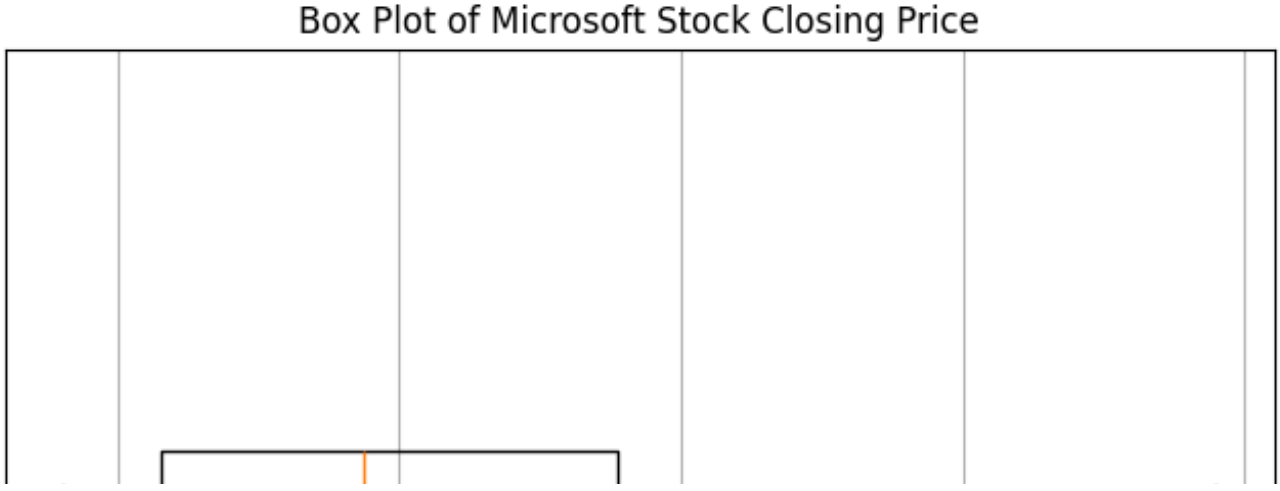
# Visualizing the correlation between the features (Open, High, Low, Close, and
```

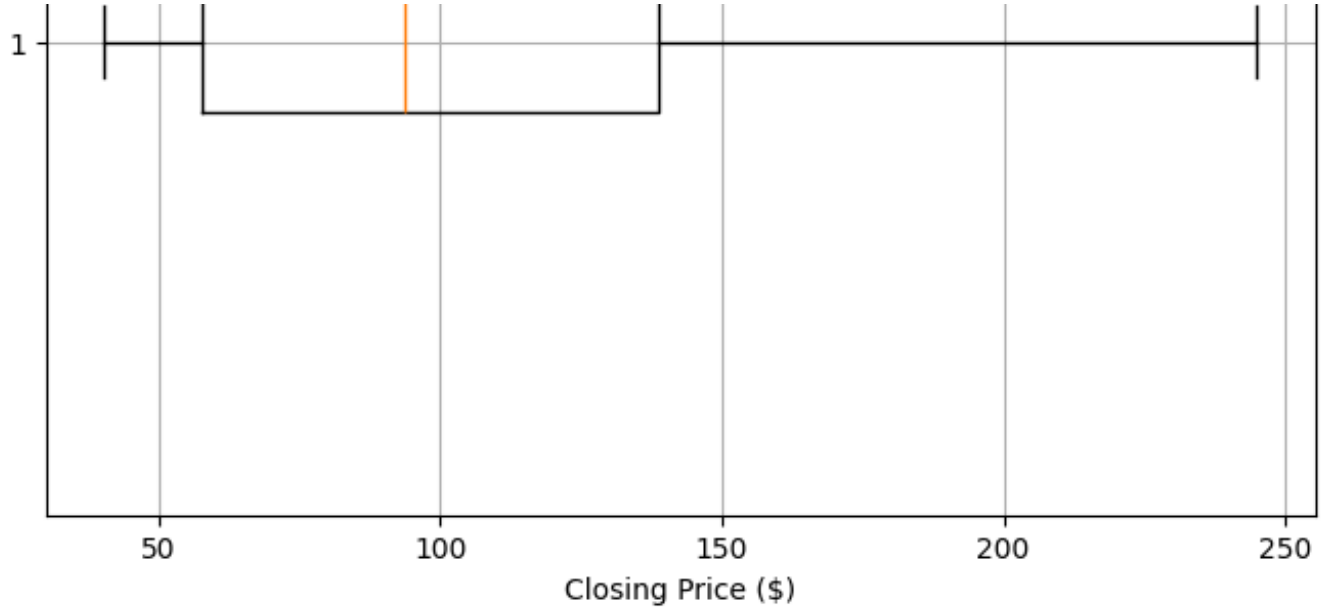
```
plt.figure(figsize=(8, 6))
correlation_matrix = df[['Open', 'High', 'Low', 'Close', 'Volume']].corr()
plt.imshow(correlation_matrix, cmap='coolwarm', interpolation='none')
plt.colorbar()
plt.title('Correlation Matrix of Microsoft Stock Features')
plt.xticks(range(len(correlation_matrix.columns)), correlation_matrix.columns,
plt.yticks(range(len(correlation_matrix.columns)), correlation_matrix.columns)
plt.show()
```

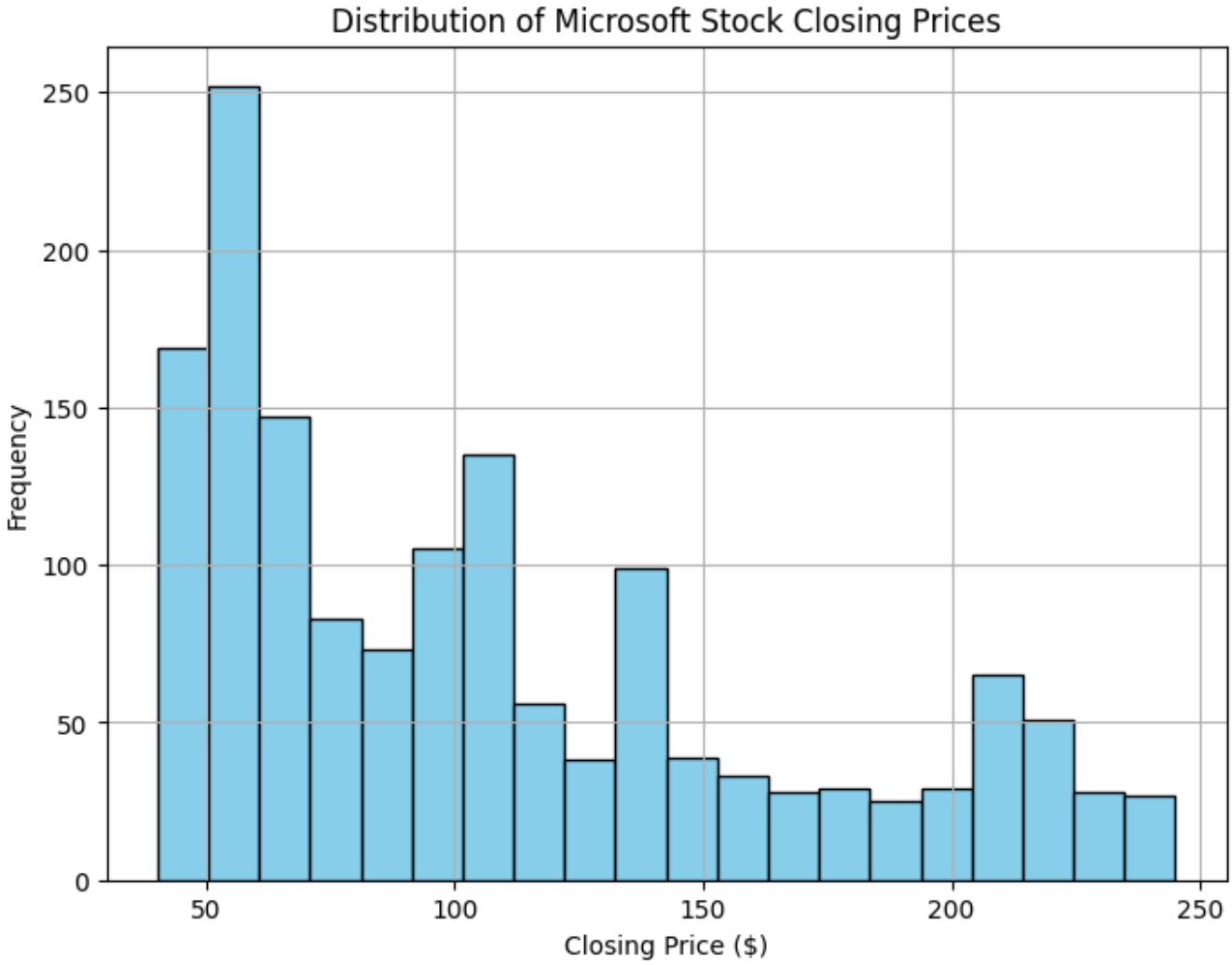
```
Descriptive Statistics:
```

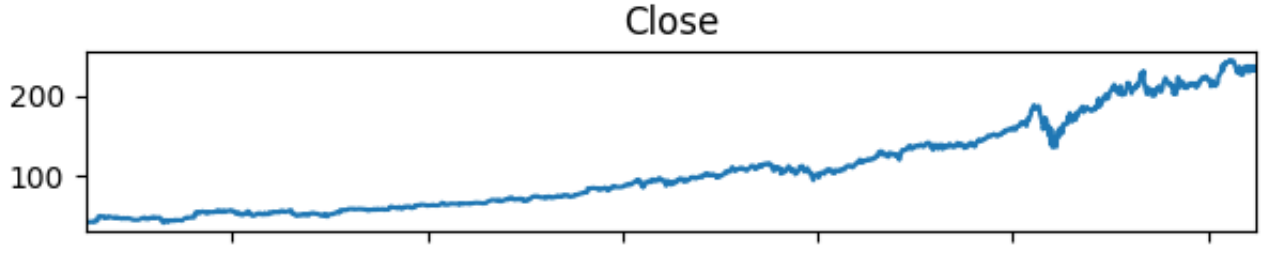
	Open	High	Low	Close	Volume
count	1511.000000	1511.000000	1511.000000	1511.000000	1.511000e+03
mean	107.385976	108.437472	106.294533	107.422091	3.019863e+07
std	56.691333	57.382276	55.977155	56.702299	1.425266e+07
min	40.340000	40.740000	39.720000	40.290000	1.016120e+05
25%	57.860000	58.060000	57.420000	57.855000	2.136213e+07
50%	93.990000	95.100000	92.920000	93.860000	2.662962e+07
75%	139.440000	140.325000	137.825000	138.965000	3.431962e+07
max	245.030000	246.130000	242.920000	244.990000	1.352271e+08

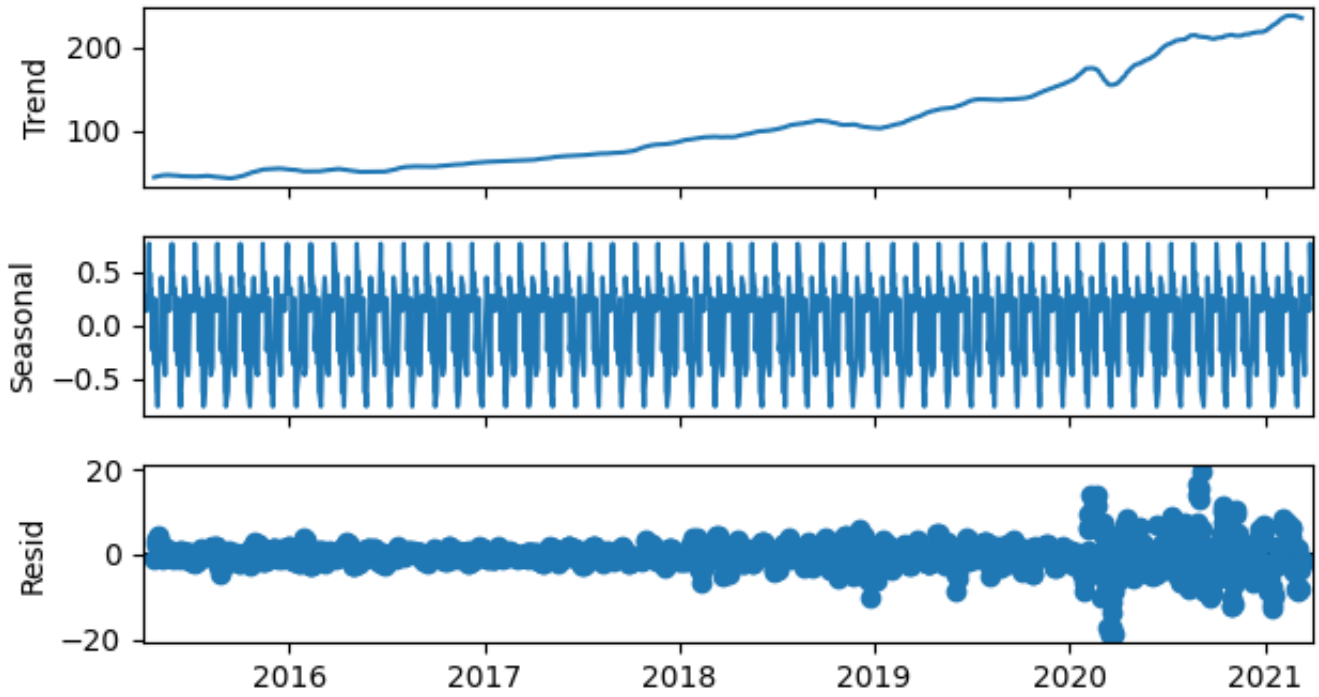


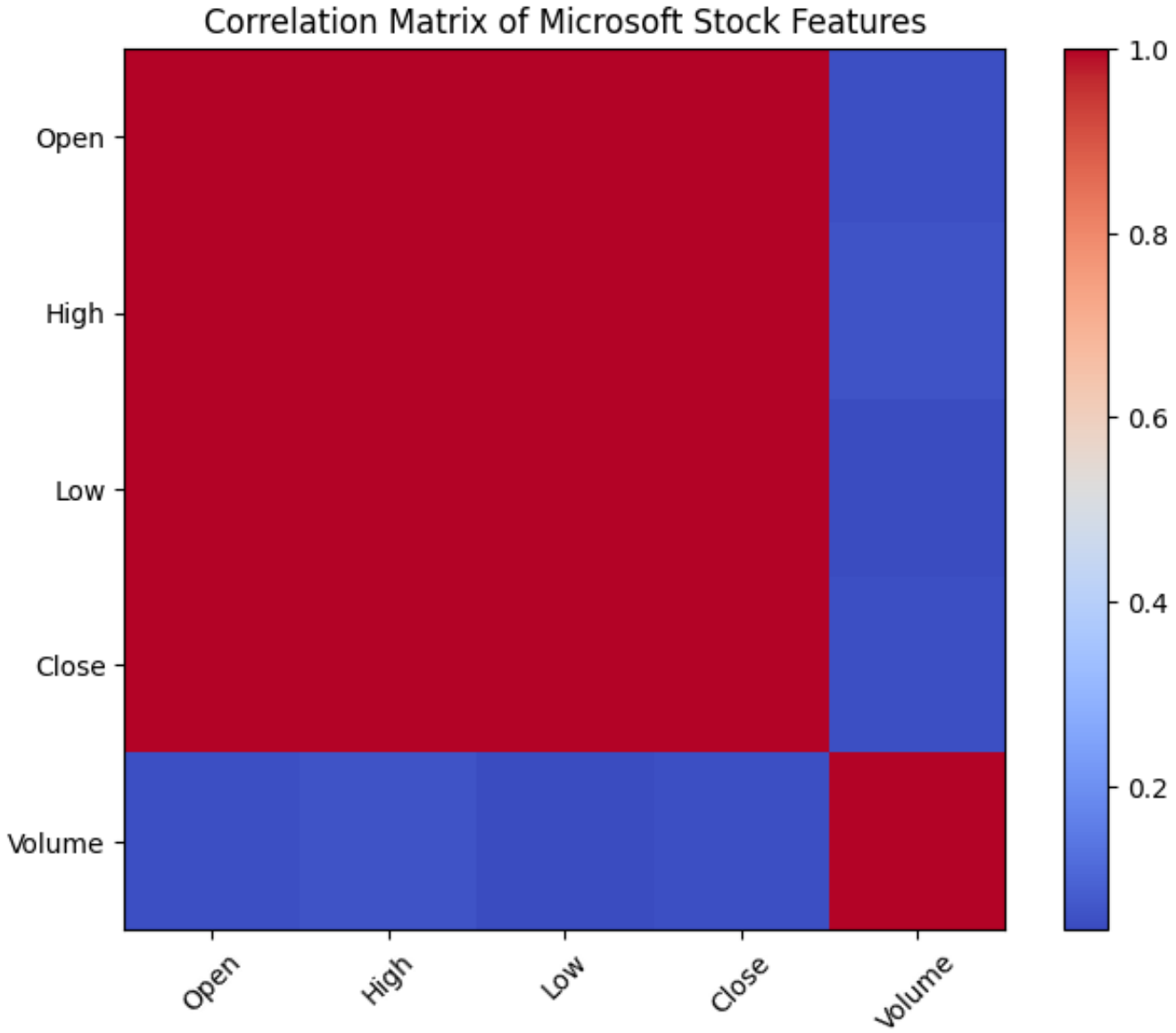



1
50
100
150
200
250
Closing Price ($)

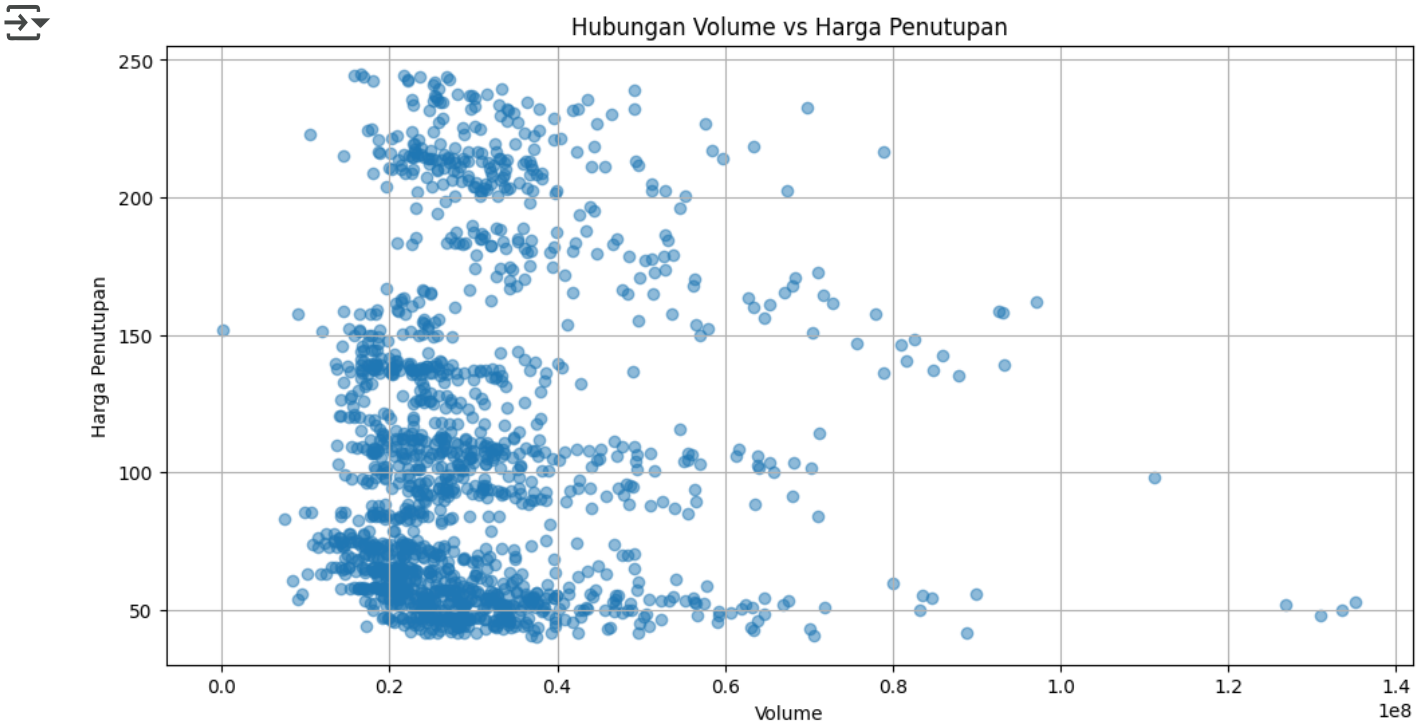

Distribution of Microsoft Stock Closing Prices
Frequency
0
50
100
150
200
250
50
100
150
200
250
Closing Price ($)


Close
200
100


Trend
200
100
Seasonal
0.5
0.0
−0.5
Resid
20
0
−20
2016
2017
2018
2019
2020
2021


Correlation Matrix of Microsoft Stock Features
Open
High
Low
Close
Volume
1.0
0.8
0.6
0.4
0.2
Open
High
Low
Close
Volume

```
# Plot hubungan antara volume dan harga penutupan
plt.figure(figsize=(12, 6))
plt.scatter(df['Volume'], df['Close'], alpha=0.5)
plt.title('Hubungan Volume vs Harga Penutupan')
plt.xlabel('Volume')
plt.ylabel('Harga Penutupan')
plt.grid(True)
plt.show()
```



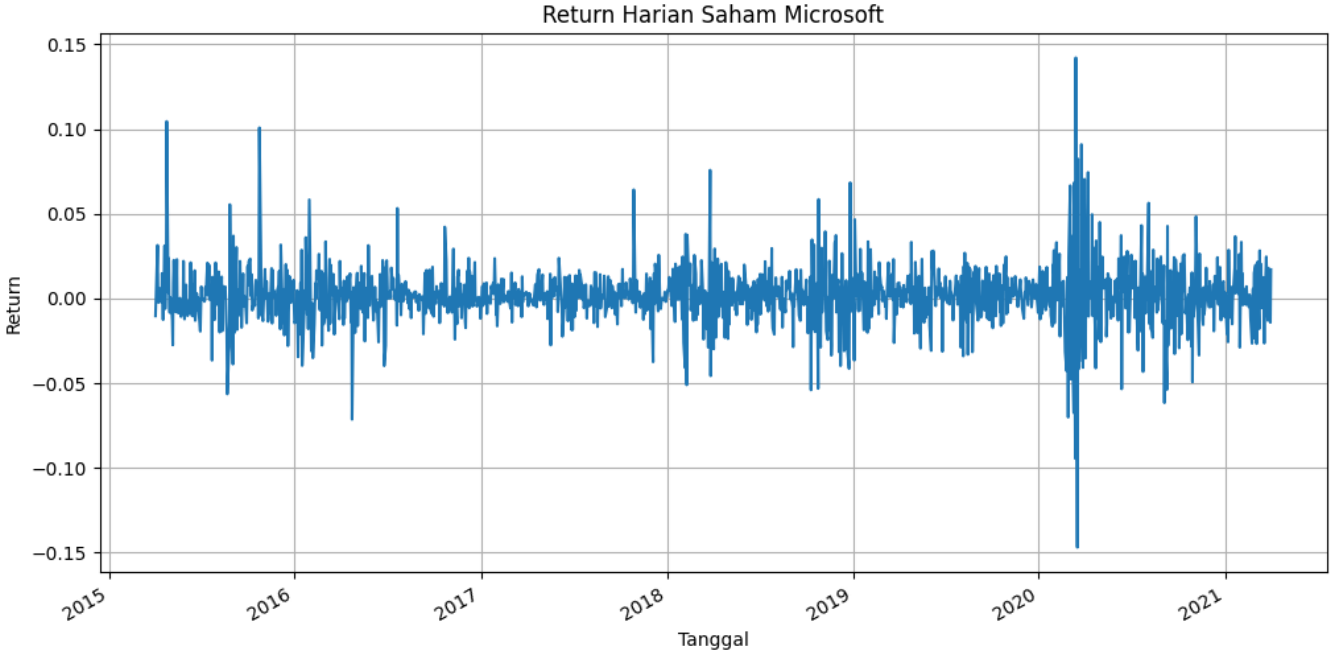

```
import seaborn as sns

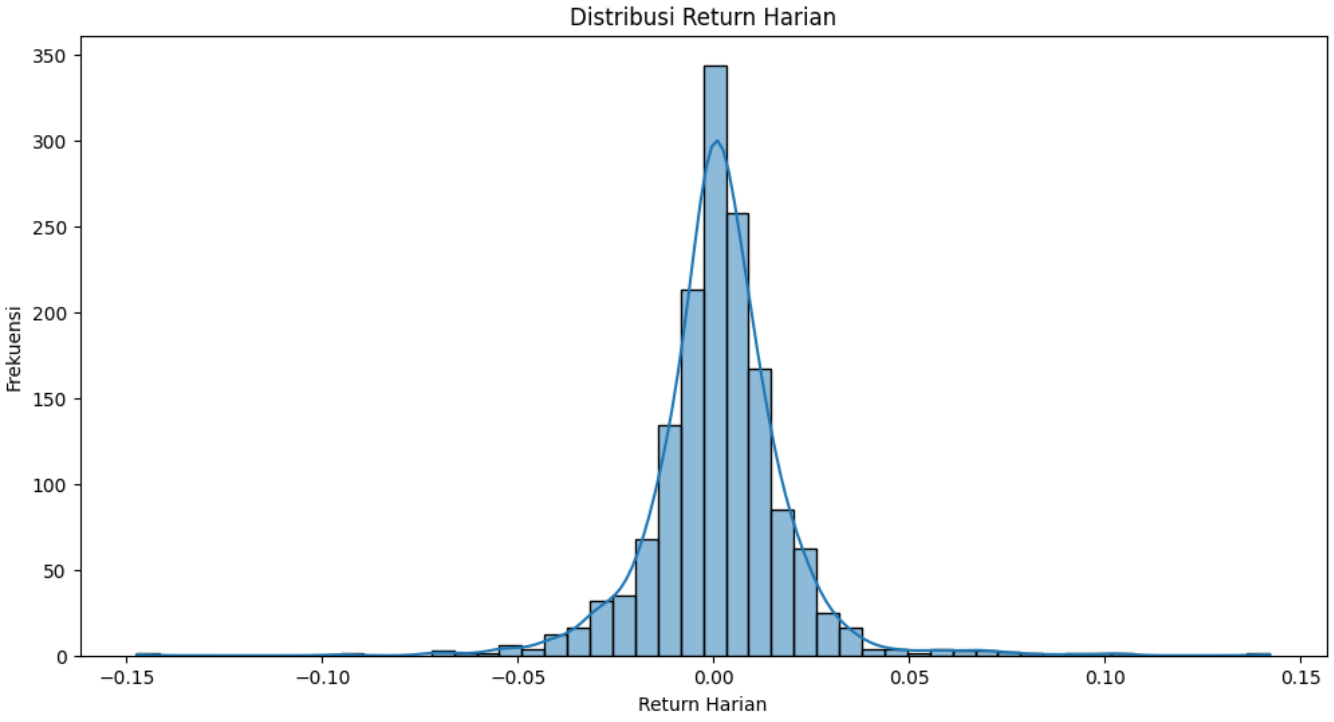
# Hitung return harian
df['Daily_Return'] = df['Close'].pct_change()

# Plot return harian
plt.figure(figsize=(12, 6))
df['Daily_Return'].plot()
```

```
plt.title('Return Harian Saham Microsoft')
plt.xlabel('Tanggal')
plt.ylabel('Return')
plt.grid(True)
plt.show()

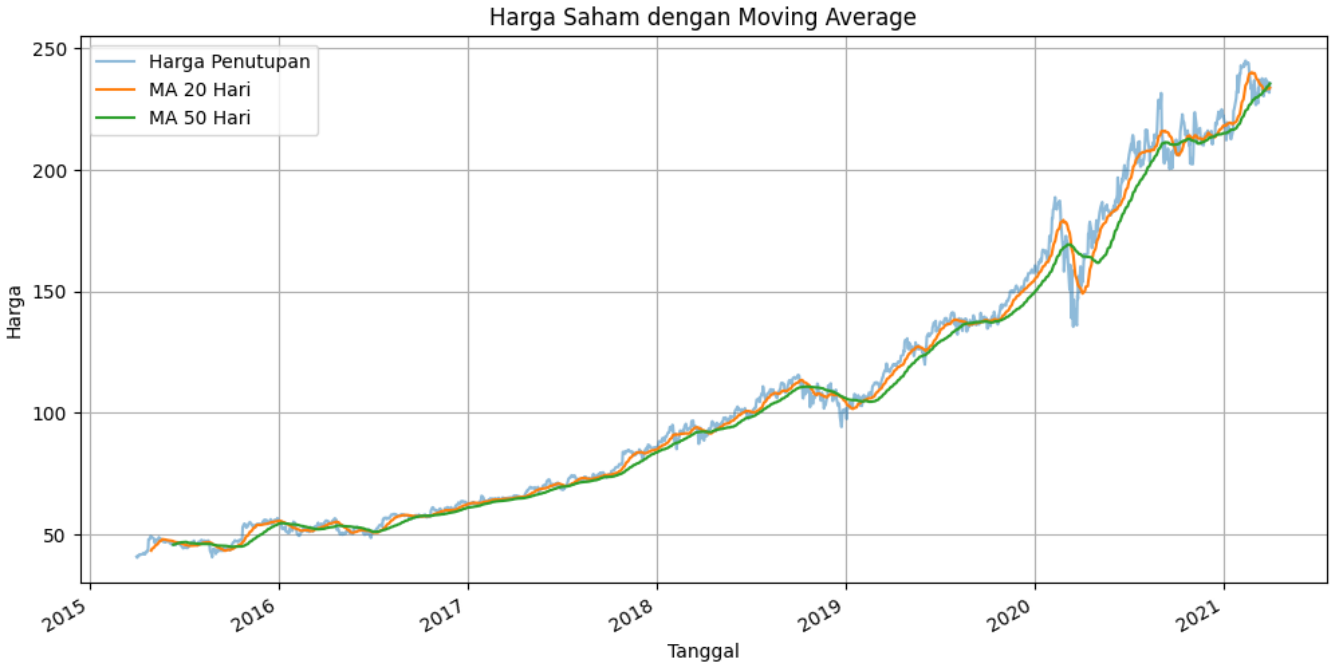
# Distribusi return harian
plt.figure(figsize=(12, 6))
sns.histplot(df['Daily_Return'].dropna(), bins=50, kde=True)
plt.title('Distribusi Return Harian')
plt.xlabel('Return Harian')
plt.ylabel('Frekuensi')
plt.show()
```





```
# Hitung moving average 20 dan 50 hari
df['MA_20'] = df['Close'].rolling(window=20).mean()
df['MA_50'] = df['Close'].rolling(window=50).mean()

# Plot harga penutupan dengan moving average
plt.figure(figsize=(12, 6))
df['Close'].plot(label='Harga Penutupan', alpha=0.5)
df['MA_20'].plot(label='MA 20 Hari')
df['MA_50'].plot(label='MA 50 Hari')
plt.title('Harga Saham dengan Moving Average')
plt.xlabel('Tanggal')
plt.ylabel('Harga')
plt.legend()
plt.grid(True)
plt.show()
```



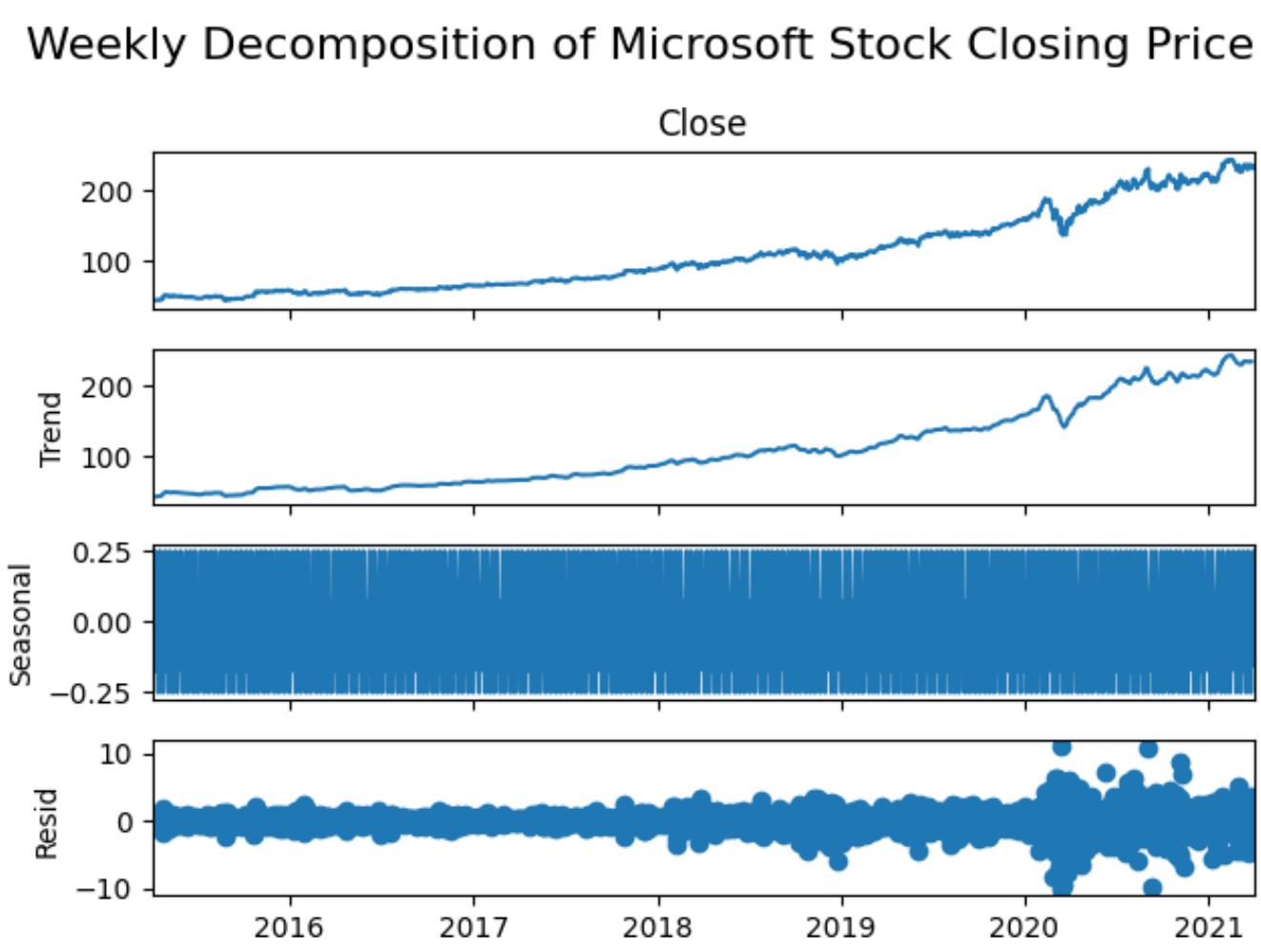
```
print(df.isnull().sum())
```

```
Open             0
High             0
Low              0
Close            0
Volume           0
Daily_Return     1
MA_20           19
MA_50           49
dtype: int64
```

Decompose the series

```
# Perform decomposition with weekly period (7 days)
decomposition = seasonal_decompose(df['Close'], model='additive', period=7)

# Plot the decomposition with a custom title
fig = decomposition.plot()
fig.suptitle('Weekly Decomposition of Microsoft Stock Closing Price', fontsize=
plt.tight_layout()  # Adjust layout to avoid overlap
plt.show()
```



Check for Stationarity

```
from statsmodels.tsa.stattools import adfuller

# Perform Augmented Dickey-Fuller test on 'Close' (Microsoft stock closing pric
result = adfuller(df['Close'])

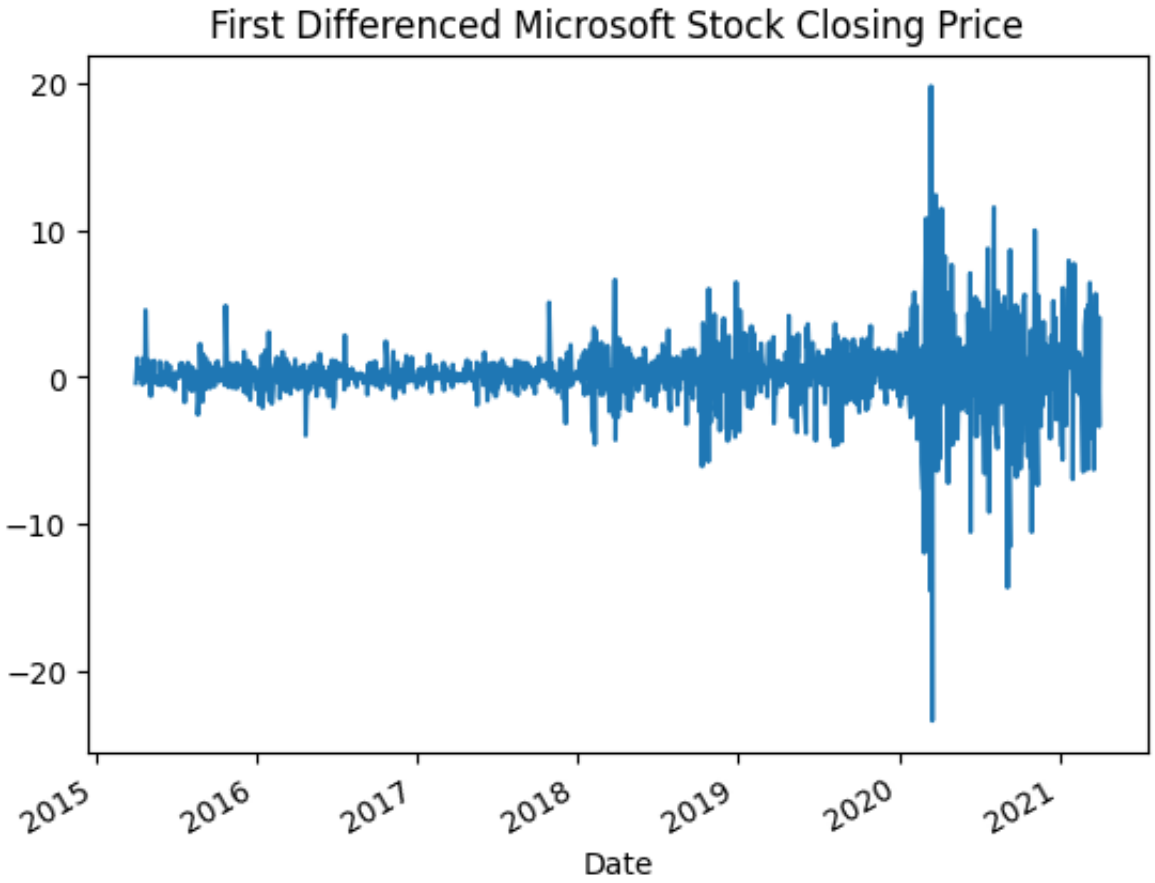
# Print the ADF Statistic and p-value
print('ADF Statistic:', result[0])
print('p-value:', result[1])

# Check if the data is stationary
if result[1] <= 0.05:
    print("Data is stationary.")
else:
    print("Data is not stationary.")
```

```
ADF Statistic: 1.7371362899270968
p-value: 0.9982158366942122
Data is not stationary.
```

```
# First differencing on 'Close' (Microsoft stock closing price)
df['Close_diff'] = df['Close'].diff()

# Plot the first differenced series
df['Close_diff'].dropna().plot(title='First Differenced Microsoft Stock Closing
plt.show()
```



```
from statsmodels.tsa.stattools import adfuller

# Perform Augmented Dickey-Fuller test on the first differenced series of 'Clos
result = adfuller(df['Close_diff'].dropna())

# Print the ADF Statistic and p-value
print(f'ADF Statistic: {result[0]}')
print(f'p-value: {result[1]}')

# Check if the data is stationary
if result[1] <= 0.05:
    print("The first differenced data is stationary.")
else:
    print("The first differenced data is not stationary.")
```

```
ADF Statistic: -10.038331065146398
p-value: 1.519593991753183e-17
The first differenced data is stationary.
```

Transformer

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam
import warnings
warnings.filterwarnings('ignore')
```

```
np.random.seed(53)
tf.random.set_seed(53)
```

```
class TransformerStockPredictor:
    def __init__(self, sequence_length=60, tf_units=50, dropout_rate=0.2):
        self.sequence_length = sequence_length
        self.tf_units = tf_units
        self.dropout_rate = dropout_rate
        self.scaler = MinMaxScaler(feature_range=(0, 1))
        self.model = None
```

```
    def load_and_preprocess_data(self, file_path):
        """Load and preprocess the stock data"""
        print("Loading data...")
        df = pd.read_csv(file_path)

        print(f"Dataset shape: {df.shape}")
        print(f"Columns: {list(df.columns)}")
        print(f"Date range: {df['Date'].min()} to {df['Date'].max()}")

        df['Date'] = pd.to_datetime(df['Date'])
        df = df.sort_values('Date').reset_index(drop=True)

        print("\nDataset Info:")
        print(df.describe())

        print(f"\nMissing values:\n{df.isnull().sum()}")

        return df

    def visualize_data(self, df):
        """Visualize the stock data"""
        plt.figure(figsize=(15, 10))

        plt.subplot(2, 2, 1)
        plt.plot(df['Date'], df['Close'])
        plt.title('Microsoft Stock Closing Price')
        plt.xlabel('Date')
        plt.ylabel('Price ($)')
        plt.xticks(rotation=45)

        plt.subplot(2, 2, 2)
        plt.plot(df['Date'], df['Volume'])
        plt.title('Microsoft Stock Volume')
        plt.xlabel('Date')
        plt.ylabel('Volume')
        plt.xticks(rotation=45)

        plt.subplot(2, 2, 3)
        plt.plot(df['Date'], df['Open'], label='Open', alpha=0.7)
        plt.plot(df['Date'], df['High'], label='High', alpha=0.7)
        plt.plot(df['Date'], df['Low'], label='Low', alpha=0.7)
        plt.plot(df['Date'], df['Close'], label='Close', alpha=0.7)
        plt.title('OHLC Prices')
        plt.xlabel('Date')
        plt.ylabel('Price ($)')
        plt.legend()
        plt.xticks(rotation=45)
```

```
        plt.subplot(2, 2, 4)
        df['MA_20'] = df['Close'].rolling(window=20).mean()
        df['MA_50'] = df['Close'].rolling(window=50).mean()
        plt.plot(df['Date'], df['Close'], label='Close', alpha=0.7)
        plt.plot(df['Date'], df['MA_20'], label='MA 20', alpha=0.7)
        plt.plot(df['Date'], df['MA_50'], label='MA 50', alpha=0.7)
        plt.title('Moving Averages')
        plt.xlabel('Date')
        plt.ylabel('Price ($)')
        plt.legend()
        plt.xticks(rotation=45)

        plt.tight_layout()
        plt.show()

    def create_sequences(self, data):
        X, y = [], []
        for i in range(self.sequence_length, len(data)):
            X.append(data[i-self.sequence_length:i, 0])
            y.append(data[i, 0])
        return np.array(X), np.array(y)

    def prepare_data(self, df, target_column='Close', train_ratio=0.8):
        """Prepare data for training"""
        print("Preparing data for training...")

        data = df[target_column].values.reshape(-1, 1)

        scaled_data = self.scaler.fit_transform(data)

        X, y = self.create_sequences(scaled_data)

        train_size = int(len(X) * train_ratio)

        X_train = X[:train_size]
        X_test = X[train_size:]
        y_train = y[:train_size]
        y_test = y[train_size:]

        X_train = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
        X_test = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))

        print(f"Training set shape: X_train: {X_train.shape}, y_train: {y_train
        print(f"Test set shape: X_test: {X_test.shape}, y_test: {y_test.shape}"

        return X_train, X_test, y_train, y_test, scaled_data

    def build_transformer_model(self):
```

```
        """Build Transformer model"""
        print("Building Transformer model...")

        input_layer = tf.keras.Input(shape=(self.sequence_length, 1))
        x = tf.keras.layers.Dense(64)(input_layer)

        attention_output = tf.keras.layers.MultiHeadAttention(num_heads=2, key_
        attention_output = tf.keras.layers.Dropout(self.dropout_rate)(attentior
        attention_output = tf.keras.layers.LayerNormalization(epsilon=1e-6)(x +

        ffn = tf.keras.Sequential([
            tf.keras.layers.Dense(64, activation='relu'),
            tf.keras.layers.Dense(64),
        ])
        ffn_output = ffn(attention_output)
        ffn_output = tf.keras.layers.Dropout(self.dropout_rate)(ffn_output)
        sequence_output = tf.keras.layers.LayerNormalization(epsilon=1e-6)(atte

        x = tf.keras.layers.GlobalAveragePooling1D()(sequence_output)
        x = tf.keras.layers.Dense(25, activation='relu')(x)
        output = tf.keras.layers.Dense(1)(x)

        self.model = tf.keras.Model(inputs=input_layer, outputs=output)

        # ❗ INI WAJIB DITAMBAHKAN
        self.model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
            loss='mse',
            metrics=['mae', 'mse']
        )

        self.model.summary()
        return self.model



    def train_model(self, X_train, y_train, X_test, y_test, epochs=100, batch_s
        print("Training model...")

        early_stopping = tf.keras.callbacks.EarlyStopping(
            monitor='val_loss', patience=10, restore_best_weights=True
        )

        reduce_lr = tf.keras.callbacks.ReduceLROnPlateau(
            monitor='val_loss', factor=0.2, patience=5, min_lr=0.0001
        )

        history = self.model.fit(
```

```
            X_train, y_train,
            epochs=epochs,
            batch_size=batch_size,
            validation_split=validation_split,
            callbacks=[early_stopping, reduce_lr],
            verbose=1
        )

        return history

    def evaluate_model(self, X_test, y_test):
        """Evaluate the model"""
        print("Evaluating model...")

        predictions = self.model.predict(X_test)

        predictions = self.scaler.inverse_transform(predictions)
        y_test_actual = self.scaler.inverse_transform(y_test.reshape(-1, 1))

        mse = mean_squared_error(y_test_actual, predictions)
        mae = mean_absolute_error(y_test_actual, predictions)
        rmse = np.sqrt(mse)
        r2 = r2_score(y_test_actual, predictions)

        print(f"Model Performance Metrics:")
        print(f"MSE: {mse:.4f}")
        print(f"MAE: {mae:.4f}")
        print(f"RMSE: {rmse:.4f}")
        print(f"R² Score: {r2:.4f}")

        return predictions, y_test_actual, {'MSE': mse, 'MAE': mae, 'RMSE': rms

    def plot_training_history(self, history):
        """Plot training history"""
        plt.figure(figsize=(12, 4))

        plt.subplot(1, 2, 1)
        plt.plot(history.history['loss'], label='Training Loss')
        plt.plot(history.history['val_loss'], label='Validation Loss')
        plt.title('Model Loss')
        plt.ylabel('Loss')
        plt.xlabel('Epoch')
        plt.legend()

        plt.subplot(1, 2, 2)
        plt.plot(history.history['mae'], label='Training MAE')
        plt.plot(history.history['val_mae'], label='Validation MAE')
        plt.title('Model MAE')
```

```
        plt.ylabel('MAE')
        plt.xlabel('Epoch')
        plt.legend()

        plt.tight_layout()
        plt.show()

    def plot_predictions(self, predictions, y_test_actual, df, train_ratio=0.8)
        """Plot predictions vs actual values"""
        plt.figure(figsize=(15, 8))

        split_point = int(len(df) * train_ratio)
        test_dates = df['Date'].iloc[split_point + self.sequence_length:]

        plt.subplot(2, 1, 1)
        plt.plot(df['Date'], df['Close'], label='Full Historical Data', alpha=0
        plt.plot(test_dates, predictions, label='Predictions', color='red', lin
        plt.plot(test_dates, y_test_actual, label='Actual Test Data', color='gr
        plt.title('Microsoft Stock Price Prediction - Full View')
        plt.xlabel('Date')
        plt.ylabel('Price ($)')
        plt.legend()
        plt.xticks(rotation=45)

        plt.subplot(2, 1, 2)
        plt.plot(test_dates, predictions, label='Predictions', color='red', lin
        plt.plot(test_dates, y_test_actual, label='Actual', color='green', line
        plt.title('Predictions vs Actual (Test Period Only)')
        plt.xlabel('Date')
        plt.ylabel('Price ($)')
        plt.legend()
        plt.xticks(rotation=45)

        plt.tight_layout()
        plt.show()

        residuals = y_test_actual.flatten() - predictions.flatten()
        plt.figure(figsize=(12, 4))

        plt.subplot(1, 2, 1)
        plt.scatter(predictions, residuals, alpha=0.7)
        plt.xlabel('Predicted Values')
        plt.ylabel('Residuals')
        plt.title('Residuals vs Predicted Values')
        plt.axhline(y=0, color='red', linestyle='--')

        plt.subplot(1, 2, 2)
        plt.hist(residuals, bins=30, alpha=0.7)
```

```
        plt.xlabel('Residuals')
        plt.ylabel('Frequency')
        plt.title('Distribution of Residuals')

        plt.tight_layout()
        plt.show()

    def predict_future(self, df, days=30):
        """Predict future stock prices"""
        print(f"Predicting next {days} days...")

        last_sequence = df['Close'].values[-self.sequence_length:]
        last_sequence_scaled = self.scaler.transform(last_sequence.reshape(-1,

        future_predictions = []
        current_sequence = last_sequence_scaled.copy()

        for _ in range(days):
            X_pred = current_sequence.reshape((1, self.sequence_length, 1))

            next_pred = self.model.predict(X_pred, verbose=0)
            future_predictions.append(next_pred[0, 0])

            current_sequence = np.roll(current_sequence, -1)
            current_sequence[-1] = next_pred

        future_predictions = np.array(future_predictions).reshape(-1, 1)
        future_predictions = self.scaler.inverse_transform(future_predictions)

        last_date = pd.to_datetime(df['Date'].iloc[-1])
        future_dates = pd.date_range(start=last_date + pd.Timedelta(days=1), pe

        plt.figure(figsize=(12, 6))
        plt.plot(df['Date'].tail(100), df['Close'].tail(100), label='Historical
        plt.plot(future_dates, future_predictions, label=f'Future Predictions (
                color='red', linewidth=2, marker='o', markersize=4)
        plt.title(f'Microsoft Stock Price - Future Predictions')
        plt.xlabel('Date')
        plt.ylabel('Price ($)')
        plt.legend()
        plt.xticks(rotation=45)
        plt.tight_layout()
        plt.show()

        return future_predictions, future_dates
```

```
def main():
    file_path = 'Microsoft_Stock.csv'

    try:
        predictor = TransformerStockPredictor()
        df = predictor.load_and_preprocess_data(file_path)

        predictor.visualize_data(df)
        X_train, X_test, y_train, y_test, scaled_data = predictor.prepare_data(
        model = predictor.build_transformer_model()
        history = predictor.train_model(X_train, y_train, X_test, y_test, epoch
        predictor.plot_training_history(history)
        predictions, y_test_actual, metrics = predictor.evaluate_model(X_test,
        predictor.plot_predictions(predictions, y_test_actual, df)
        future_pred, future_dates = predictor.predict_future(df, days=30)

        print("\nFuture Predictions Summary:")
        for i, (date, price) in enumerate(zip(future_dates, future_pred)):
            print(f"Day {i+1} ({date.strftime('%Y-%m-%d')}): ${price[0]:.2f}")

        return predictor, df, metrics

    except FileNotFoundError:
        print(f"Error: File '{file_path}' not found.")
        print("Please make sure you have downloaded the dataset and updated the
        print("\nTo use this code:")
        print("1. Download the dataset from the Kaggle link")
        print("2. Update the 'file_path' variable with the correct path to your
        print("3. Run the script again")

        return None, None, None
```

```
if __name__ == "__main__":
    predictor, df, metrics = main()

    if predictor is not None:
        print("\n" + "="*50)
        print("="*50)
        print("\nModel Architecture:")
        print("- Dropout layers for regularization")
        print("- Dense output layer")
        print("- Sequence length: 60 days")
        print("\nNext steps for improvement:")
        print("1. Experiment with different architectures")
        print("2. Try different hyperparameters")
        print("3. Add more features (technical indicators)")
        print("4. Use ensemble methods")
        print("5. Implement attention mechanisms")
```

```
Loading data...
Error: File 'Microsoft_Stock.csv' not found.
Please make sure you have downloaded the dataset and updated the file path.

To use this code:
1. Download the dataset from the Kaggle link
2. Update the 'file_path' variable with the correct path to your CSV file
3. Run the script again
```